HTML

Complete Foundations Study Notes

The Odin Project • Foundations

Exam Preparation · Concepts · Code Examples · Practical Exercises

Repository: The Odin Project – Web Dev Learning
Suggested location: Foundations/04-HTML/notes.md

Learn the structure first. Style it later. Make it interactive after that.

HTML – Complete Study Notes

The Odin Project | Foundations

Subject: HyperText Markup Language (HTML)

Learning resource: The Odin Project

Level: Beginner / Foundations

Purpose: Exam preparation, revision and practical reference

Table of Contents

1. Introduction to HTML
2. HTML vs CSS vs JavaScript
3. HTML Elements and Tags
4. HTML Attributes
5. HTML Boilerplate
6. HTML Document Structure
7. Working with Text
8. HTML Headings and Paragraphs
9. Strong and Emphasis
10. Nesting, Indentation and Comments
11. HTML Lists
12. HTML Links
13. Absolute and Relative URLs
14. HTML Images
15. File Paths
16. Semantic HTML (additional knowledge)
17. Validation and Common Errors
18. The Odin Project: Recipes Project
19. Important HTML Tags Reference
20. Exam Preparation Questions
21. Practical Exercises
22. Quick Revision Sheet

1. Introduction to HTML

1.1 What is HTML?

HTML stands for **HyperText Markup Language**. It is the standard markup language used to create and structure content on webpages. HTML tells the browser what content exists and how that content is organized.

HTML can define headings, paragraphs, images, links, lists, tables, forms and other page content.

```
<h1>Welcome to My Website</h1>
<p>This is my first webpage.</p>
```

The browser reads the HTML and displays the content according to the meaning of its elements.

1.2 Why is HTML important?

HTML provides the structure of a webpage. For a recipe page, for example:

- A heading represents the recipe name.
- A paragraph describes the recipe.
- An unordered list contains ingredients.
- An ordered list contains cooking instructions.
- An image displays the finished food.
- A link connects the recipe to another page.

1.3 Is HTML a programming language?

No. HTML is a **markup language**, not a programming language. Programming languages implement logic such as calculations, conditions and loops. HTML primarily describes the structure and meaning of content; it does not independently provide programming logic.

2. HTML vs CSS vs JavaScript

These technologies work together to create modern websites.

Technology	Full form	Main responsibility
HTML	HyperText Markup Language	Structure and content
CSS	Cascading Style Sheets	Appearance and styling
JavaScript	JavaScript	Behaviour and interactivity

HTML – Structure

```
<h1>My Portfolio</h1>
<p>Welcome to my portfolio website.</p>
<button>Contact Me</button>
```

CSS – Styling

```
h1 {
    color: blue;
    font-size: 36px;
}

p {
    color: gray;
}
```

JavaScript – Behaviour

```
const button = document.querySelector("button");

button.addEventListener("click", function () {
    alert("Hello!");
});
```

Easy analogy: HTML is the structure of a house, CSS is its paint and decoration, and JavaScript is its interactive functionality and systems.

3. HTML Elements and Tags

3.1 What is an HTML tag?

A tag is a keyword enclosed in angle brackets. For example, `<p>` is an opening paragraph tag.

3.2 What is an HTML element?

An HTML element generally consists of an opening tag, content and a closing tag.

```
<p>Hello World!</p>
```

- `<p>` — opening tag
- `Hello World!` — content
- `</p>` — closing tag
- The complete structure — HTML element

Tag	Element
A markup instruction enclosed in angle brackets	The complete HTML structure
Example: `<p>`	Example: `<p>Hello</p>`

3.3 Opening and closing tags

An opening tag marks the beginning of an element: `<h1>`. A closing tag marks its end and includes a forward slash: `</h1>`.

3.4 Void elements

Void elements do not contain content and do not have closing tags.

```
<br>
<hr>
<img src="image.jpg" alt="A landscape">
<meta charset="UTF-8">
```

Common void elements include:

- `<br>` — line break
- `<hr>` — thematic break
- `<img>` — embeds an image
- `<meta>` — document metadata
- `<input>` — input control
- `<link>` — links an external resource
- `<source>` — media source
- `<wbr>` — suggested word-break opportunity

Important: Void elements must not have closing tags. `<br />` may appear in examples, but the trailing slash is unnecessary in HTML.

4. HTML Attributes

4.1 What is an attribute?

An HTML attribute provides additional information about an element. Attributes are written inside the opening tag.

```
<a href="https://www.google.com">Google</a>
```

- `<a>` — anchor element
- `href` — attribute name
- URL — attribute value
- `Google` — visible link text

General syntax:

```
<tagname attribute="value">Content</tagname>
```

Example with multiple attributes:

```
<img
    src="images/cat.jpg"
    alt="A brown cat sitting on a chair"
    width="300"
>
```

4.2 Common attributes

Attribute	Purpose
`href`	Specifies a link destination
`src`	Specifies a resource location
`alt`	Alternative text for an image
`id`	Unique identifier within a document
`class`	Assigns one or more class names
`title`	Additional advisory information
`lang`	Specifies the language
`width`	Specifies width
`height`	Specifies height
`target`	Specifies where to open a linked resource
`rel`	Describes the relationship to a linked resource

Example:

```
<p id="intro" class="description">
    Welcome to my website.
</p>
```

`id` values should be unique within an HTML document. Multiple elements can share a class.

5. HTML Boilerplate

5.1 What is HTML boilerplate?

HTML boilerplate is the standard starting structure of an HTML document. It provides the browser with document information and establishes the page structure.

5.2 Standard HTML boilerplate

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Webpage</title>
</head>

<body>
    <h1>Hello World!</h1>
</body>

</html>
```

5.3 DOCTYPE declaration

```
<!DOCTYPE html>
```

This tells the browser to interpret the document using modern HTML standards. It is written at the beginning, is not an HTML element and has no closing tag. It helps the browser use standards mode rather than quirks mode.

5.4 HTML root element

```
<html lang="en">
</html>
```

The `<html>` element is the root element. All other HTML elements are contained within it, except the DOCTYPE declaration. The `lang` attribute specifies the document's primary language. `en` means English; `ur` means Urdu.

5.5 Head element

```
<head>
    <meta charset="UTF-8">
    <title>My Website</title>
</head>
```

The `<head>` contains metadata and information for the browser and other systems. It does not contain the normal visible webpage content. Common children include `<meta>`, `<title>`, `<link>`, `<style>` and appropriately placed `<script>` elements.

5.6 Character encoding

```
<meta charset="UTF-8">
```

This specifies the document's character encoding. UTF-8 supports characters and symbols from many languages.

5.7 Viewport metadata

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

This helps browsers display a page correctly on different screen sizes. `width=device-width` sets the viewport width to the device width; `initial-scale=1.0` sets the initial zoom level.

5.8 Title element

```
<title>My First Webpage</title>
```

The title normally appears in the browser tab and may be used in bookmarks and search result titles. It belongs inside `<head>`.

5.9 Body element

```
<body>
    <h1>Hello World!</h1>
    <p>This is my first webpage.</p>
</body>
```

The body contains the webpage's main content, such as headings, paragraphs, images, links and lists.

Head	Body
Contains document metadata	Contains page content
Includes title and meta information	Includes headings, paragraphs, images, etc.
Not normally displayed as page content	Displays content in the browser

5.10 Why is the homepage often named index.html?

Web servers commonly look for `index.html` as the default document when someone visits a website or directory.

```
my-website/
■■■ index.html
■■■ about.html
■■■ contact.html
```

6. HTML Document Structure and the DOM

HTML elements form a tree-like hierarchy.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My Website</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>This is a paragraph.</p>
</body>
</html>
```

Conceptual tree:

```
Document
■■■ DOCTYPE
■■■ html
    ■■■ head
    ■   ■■■ meta
    ■   ■■■ title
    ■■■ body
        ■■■ h1
        ■■■ p
```

The browser represents the parsed document as the **Document Object Model (DOM)**. JavaScript can use the DOM to access and modify page elements.

7. Working with Text

HTML provides elements to organize and give meaning to text. Important Foundations elements include:

- `<h1>` through `<h6>`
- `<p>`
- `<strong>`
- `<em>`

8. Headings and Paragraphs

8.1 HTML headings

HTML provides six heading levels.

```
<h1>Main Heading</h1>
<h2>Second-Level Heading</h2>
<h3>Third-Level Heading</h3>
<h4>Fourth-Level Heading</h4>
<h5>Fifth-Level Heading</h5>
<h6>Sixth-Level Heading</h6>
```

Tag	Description
`<h1>`	Highest-level heading
`<h2>`	Second-level heading
`<h3>`	Third-level heading
`<h4>`	Fourth-level heading
`<h5>`	Fifth-level heading
`<h6>`	Lowest-level heading

Example hierarchy:

```
<h1>Web Development</h1>
<h2>Frontend Development</h2>
<h3>HTML</h3>
<h3>CSS</h3>
<h2>Backend Development</h2>
<h3>Node.js</h3>
```

Headings organize content, make it easier to understand, help users navigate and provide structure for assistive technologies. Choose heading levels according to document hierarchy, not simply because of their default font size. CSS controls appearance.

8.2 Paragraphs

The `<p>` element defines a paragraph.

```
<p>HTML defines webpage structure.</p>
<p>CSS controls webpage appearance.</p>
<p>JavaScript adds interactivity.</p>
```

Source-code line breaks and repeated spaces do not automatically create separate paragraphs. Use paragraph elements to express paragraph structure.

9. Strong and Emphasis

9.1 Strong element

`<strong>` indicates strong importance, seriousness or urgency.

```
<p><strong>Warning:</strong> Save your work before closing.</p>
```

Browsers generally display it in bold.

9.2 Emphasis element

`<em>` indicates stress emphasis.

```
<p>You <em>must</em> complete the assignment.</p>
```

Browsers generally display it in italics.

`<strong>`	`<em>`
Indicates strong importance	Indicates stress emphasis
Usually appears bold	Usually appears italic
Communicates importance	Communicates emphasis

The `<b>` element draws attention without necessarily indicating importance. The `<i>` element represents text in an alternate voice or mood, or otherwise distinguished from normal prose. Use `<strong>` and `<em>` when their meanings are intended.

10. Nesting, Indentation and Comments

10.1 Nesting

Nesting means placing one HTML element inside another.

```
<p>
    This is a paragraph containing
    <strong>important text</strong>.
</p>
```

Here, `<strong>` is nested inside `<p>`.

10.2 Parent, child and sibling elements

In this example:

```
<body>
    <p>Hello World!</p>
</body>
```

- `<body>` is the parent of `<p>`.

- `<p>` is a child of `<body>`.
- Elements that share the same parent are siblings.

```
<body>
    <h1>Welcome</h1>
    <p>First paragraph.</p>
    <p>Second paragraph.</p>
</body>
```

The heading and both paragraphs are siblings.

10.3 Proper nesting

Correct:

```
<p>This is <strong>important</strong> information.</p>
```

Incorrect:

```
<p>This is <strong>important</p></strong>
```

Elements must be nested according to their permitted content models and closed in the correct order.

10.4 Indentation

Indentation uses spaces to show the hierarchy of elements.

```
<body>
    <h1>My Website</h1>
    <div>
        <p>First paragraph.</p>
        <p>Second paragraph.</p>
    </div>
</body>
```

Consistent indentation, commonly two or four spaces, makes code easier to read and maintain.

10.5 HTML comments

Comments add notes to source code and are not displayed as normal webpage content.

```
<!-- This is an HTML comment -->

<body>
    <!-- Main heading -->
    <h1>My Website</h1>
</body>
```

Comments start with `<!--` and end with `-->`. They are visible to anyone who inspects the page source, so never put passwords, API keys or private information in them.

11. HTML Lists

HTML provides different list elements to organize information. The two basic list types are unordered lists and ordered lists. Both use `<li>` for individual list items.

11.1 Unordered lists

An unordered list is used when the order of items does not matter. It uses `<ul>` and `<li>`.

```
<h2>Shopping List</h2>
<ul>
    <li>Milk</li>
    <li>Bread</li>
    <li>Eggs</li>
    <li>Rice</li>
</ul>
```

Unordered lists are useful for ingredients, shopping lists, categories and features. Browsers normally display bullet points.

11.2 Ordered lists

An ordered list is used when the sequence matters. It uses `<ol>` and `<li>`.

```
<h2>Cooking Instructions</h2>
<ol>
    <li>Prepare the ingredients.</li>
    <li>Heat the pan.</li>
    <li>Add the ingredients.</li>
    <li>Cook until ready.</li>
</ol>
```

Ordered lists are useful for instructions, tutorials, rankings and sequential processes.

11.3 Ordered list attributes

The `start` attribute changes the starting number:

```
<ol start="5">
    <li>Fifth item</li>
    <li>Sixth item</li>
</ol>
```

The `reversed` attribute reverses the numbering direction:

```
<ol reversed>
    <li>Third</li>
    <li>Second</li>
    <li>First</li>
</ol>
```

The `type` attribute can select a numbering style:

```
<ol type="A">
    <li>First</li>
    <li>Second</li>
</ol>
```

Common `type` values: `1` for numbers, `A` for uppercase letters, `a` for lowercase letters, `I` for uppercase Roman numerals and `i` for lowercase Roman numerals. These are additional HTML features beyond the basic list lesson.

11.4 Nested lists

A nested list is placed inside a list item.

```
<ul>
    <li>Fruits
        <ul>
```

```
            <li>Apple</li>
            <li>Banana</li>
            <li>Orange</li>
        </ul>
    </li>
    <li>Vegetables
        <ul>
            <li>Potato</li>
            <li>Carrot</li>
        </ul>
    </li>
</ul>
```

You can also nest an ordered list inside an unordered list, or vice versa.

12. HTML Links

Links connect webpages and let users navigate between resources. The `<a>` element, called the **anchor element**, creates hyperlinks.

12.1 Basic link syntax

```
<a href="https://www.google.com">Visit Google</a>
```

- `<a>` — anchor element
- `href` — destination attribute
- URL — destination
- `Visit Google` — clickable text

The `href` attribute specifies the destination. Without an appropriate `href`, an anchor does not provide a normal navigational link.

12.2 External links

External links navigate to other websites.

```
<a href="https://developer.mozilla.org/">MDN Web Docs</a>
```

12.3 Internal links

Internal links connect pages within the same website.

```
website/
■■■ index.html
■■■ about.html
■■■ contact.html
```

From `index.html`:

```
<a href="about.html">About Us</a>
```

From `about.html`:

```
<a href="index.html">Home</a>
```

12.4 Opening a link in a new tab

```
<a
    href="https://www.google.com"
    target="_blank"
    rel="noopener"
>
    Open Google
</a>
```

`target="_blank"` generally opens the destination in a new tab. `rel="noopener"` prevents the opened page from accessing the original page through `window.opener`.

13. Absolute and Relative URLs

13.1 Absolute URL

An absolute URL contains the complete web address of a resource.

```
<a href="https://www.theodinproject.com/">The Odin Project</a>
```

It includes a scheme such as `https://`, a domain and possibly a path. Absolute URLs are commonly used for external websites.

13.2 Relative URL

A relative URL points to a resource based on the current document's location or the site's configured base URL.

```
<a href="about.html">About</a>
```

Relative URLs are commonly used for pages and resources within the same website.

13.3 Same directory

```
website/
■■■ index.html
■■■ about.html
```

```
<a href="about.html">About</a>
```

13.4 Child directory

```
website/
■■■ index.html
■■■ pages/
    ■■■ about.html
```

From `index.html`:

```
<a href="pages/about.html">About</a>
```

13.5 Parent directory

From `pages/about.html`, link to the homepage:

```
<a href="../index.html">Home</a>
```

`../` moves one directory level upward.

Path	Meaning
`./`	Current directory
`../`	Parent directory
`/`	Path separator; at the start of a URL path, it indicates a root-relative path

14. HTML Images

The `<img>` element embeds an image. It is a void element.

14.1 Basic syntax

```
<img src="cat.jpg" alt="A cat">
```

- `<img>` — image element
- `src` — image source
- `alt` — alternative text

14.2 The src attribute

The `src` attribute specifies the image file location.

```
<img src="images/dog.jpg" alt="A dog">
```

The browser looks for `dog.jpg` in the `images` directory relative to the HTML document.

14.3 The alt attribute

The `alt` attribute provides alternative text describing an image. It is important for screen-reader accessibility and when an image cannot load.

```
<img
    src="images/burger.jpg"
    alt="A cheeseburger with lettuce and tomato"
>
```

For purely decorative images, empty alternative text is generally appropriate:

```
<img src="decoration.png" alt="">
```

14.4 Image dimensions

```
<img
    src="burger.jpg"
    alt="A burger"
    width="400"
    height="300"
>
```

Dimensions can help the browser reserve space before an image loads. CSS is generally preferred for flexible and responsive image sizing.

14.5 External images

```
<img
    src="https://example.com/image.jpg"
    alt="Example image"
>
```

The image must be publicly accessible for the browser to load it.

15. File Paths

File paths tell the browser where to find files such as HTML pages, images, CSS files and JavaScript files.

Example:

```
my-website/
■■■ index.html
■■■ about.html
■■■ images/
    ■■■ logo.png
    ■■■ profile.jpg
```

Inside `index.html`:

```
<img src="images/logo.png" alt="Website logo">
```

The browser starts from the directory containing `index.html`, enters `images`, finds `logo.png` and loads it.

Path	Meaning
`image.jpg`	File in the current directory
`images/image.jpg`	File inside the images directory
`../image.jpg`	File in the parent directory
`./image.jpg`	File in the current directory

Common path problems include incorrect filenames, file extensions, directory names, capitalization and the wrong number of `../` segments.

16. Semantic HTML – Additional Knowledge

Semantic HTML means using elements according to the meaning and purpose of their content.

Common semantic elements:

Element	Purpose
`<header>`	Introductory content for a page or section
`<nav>`	Navigation links
`<main>`	Main content of the document
`<section>`	Thematic grouping of content
`<article>`	Self-contained composition
`<aside>`	Complementary content

Element	Purpose
`<footer>`	Footer information

Example:

```
<body>
    <header>
        <h1>My Website</h1>
    </header>

    <nav>
        <a href="index.html">Home</a>
        <a href="about.html">About</a>
    </nav>

    <main>
        <section>
            <h2>Introduction</h2>
            <p>Welcome to my website.</p>
        </section>
    </main>

    <footer>
        <p>Copyright 2026</p>
    </footer>
</body>
```

Semantic HTML improves document structure, accessibility and maintainability. These elements are supplementary knowledge beyond the initial HTML Foundations lessons.

17. HTML Validation and Common Errors

HTML validation checks whether markup follows HTML syntax and conformance rules. A validator can identify incorrect nesting, invalid elements or attributes, missing required attributes and other errors.

W3C HTML Validator: https://validator.w3.org/

You can validate by URL, file upload or pasted code.

17.1 Common errors

Missing closing tag

Incorrect:

```
<p>Hello World!
```

Correct:

```
<p>Hello World!</p>
```

Incorrect nesting

Incorrect:

```
<p><strong>Hello</p></strong>
```

Correct:

```
<p><strong>Hello</strong></p>
```

Missing or incorrect link destination

Incorrect:

```
<a href>Google</a>
```

Correct:

```
<a href="https://www.google.com">Google</a>
```

Missing image alternative text

```
<img src="cat.jpg" alt="A cat">
```

Incorrect file path

If the image is in an `images` directory, use:

```
<img src="images/burger.jpg" alt="Burger">
```

Other common problems include incomplete boilerplate, placing visible content inside `<head>`, forgetting a title and assuming source-code whitespace always appears the same in the browser.

18. The Odin Project: Recipes Project

The Recipes project applies HTML knowledge through a basic recipe website.

18.1 Project objectives

Create a homepage and individual recipe pages containing descriptions, ingredients, instructions, images and links.

18.2 Recommended structure

```
odin-recipes/
■■■ index.html
■■■ recipes/
■   ■■■ burger.html
■   ■■■ pizza.html
■   ■■■ pasta.html
■■■ images/
    ■■■ burger.jpg
    ■■■ pizza.jpg
    ■■■ pasta.jpg
```

18.3 Homepage example

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Odin Recipes</title>
</head>
<body>
    <h1>Odin Recipes</h1>

    <ul>
        <li><a href="recipes/burger.html">Burger Recipe</a></li>
        <li><a href="recipes/pizza.html">Pizza Recipe</a></li>
        <li><a href="recipes/pasta.html">Pasta Recipe</a></li>
    </ul>
</body>
</html>
```

18.4 Individual recipe page example

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Burger Recipe</title>
</head>
<body>
    <h1>Homemade Burger</h1>

    <img src="../images/burger.jpg" alt="Homemade burger">

    <h2>Description</h2>
    <p>A homemade burger prepared with a bun, vegetables, cheese and a burger patty.</p>

    <h2>Ingredients</h2>
    <ul>
        <li>Burger bun</li>
        <li>Cheese</li>
        <li>Lettuce</li>
        <li>Tomato</li>
        <li>Burger patty</li>
    </ul>

    <h2>Steps</h2>
    <ol>
        <li>Prepare the ingredients.</li>
        <li>Prepare the burger patty.</li>
        <li>Arrange the ingredients inside the bun.</li>
        <li>Serve the burger.</li>
    </ol>

    <a href="../index.html">Back to Homepage</a>
</body>
</html>
```

18.5 Concepts applied

Concept	Project use
Boilerplate	Every HTML page
Headings	Recipe names and sections
Paragraphs	Recipe descriptions
Unordered lists	Ingredients
Ordered lists	Cooking instructions
Anchor elements	Navigation
Images	Recipe pictures
Relative paths	Connecting pages and images

18.6 Learning outcomes

By completing the project, you practise creating multiple pages, structuring content, writing lists, linking webpages, adding images, organizing directories and using relative file paths. The project can be built without CSS.

19. Important HTML Tags Reference

Tag	Name	Purpose
`<!DOCTYPE html>`	Doctype	Declares HTML document type
`<html>`	HTML	Root element
`<head>`	Head	Document metadata
`<body>`	Body	Page content
`<title>`	Title	Browser tab title
`<meta>`	Meta	Document metadata
`<h1>–<h6>`	Headings	Heading levels
`<p>`	Paragraph	Paragraph text
`<strong>`	Strong	Important text
`<em>`	Emphasis	Stressed text
`<b>`	Bold	Stylistically distinguished text
`<i>`	Italic	Alternate voice or mood
` `	Break	Line break
`<hr>`	Thematic break	Thematic separation
`<ul>`	Unordered list	Bulleted list
`<ol>`	Ordered list	Numbered list
`<li>`	List item	Individual list entry
`<a>`	Anchor	Hyperlink
`<img>`	Image	Embeds an image
`<div>`	Division	Generic block container
`<span>`	Span	Generic inline container
`<header>`	Header	Introductory content
`<nav>`	Navigation	Navigation section
`<main>`	Main	Main page content
`<section>`	Section	Thematic grouping
`<article>`	Article	Self-contained content
`<footer>`	Footer	Footer information

20. Exam Preparation Questions

Try answering without looking at the notes.

Part A: Basic concepts

1. What does HTML stand for?
2. What is the primary purpose of HTML?
3. Is HTML a programming language? Explain.
4. What is the difference between HTML, CSS and JavaScript?

5. What is an HTML tag?
6. What is an HTML element?
7. Explain the difference between an opening tag and a closing tag.
8. What are void elements? Give five examples.
9. What are HTML attributes?
10. What is the purpose of the `href` attribute?

Part B: HTML boilerplate

11. What is HTML boilerplate?
12. What is the purpose of `<!DOCTYPE html>`?
13. What is the root element of an HTML document?
14. What is the purpose of `<head>`?
15. What is the purpose of `<body>`?
16. What does `<meta charset="UTF-8">` do?
17. What is the purpose of `<title>`?
18. Why is a homepage commonly named `index.html`?
19. What is the purpose of the `lang` attribute?
20. What does the viewport meta tag do?

Part C: Text and lists

21. How many heading levels are available in HTML?
22. What is the difference between `<h1>` and `<h6>`?
23. Which element creates a paragraph?
24. What is the difference between `<strong>` and `<em>`?
25. What is nesting in HTML?
26. What is the difference between parent and child elements?
27. What are sibling elements?
28. What is an HTML comment?
29. Which element creates an unordered list?
30. Which element creates an ordered list?
31. What is the purpose of `<li>`?
32. When should you use an ordered list instead of an unordered list?

Part D: Links and images

33. What is an anchor element?
34. What does `href` do?
35. What is the difference between absolute and relative URLs?
36. What does `../` mean in a file path?
37. Which HTML element displays an image?
38. What is the purpose of `src`?

39. What is the purpose of `alt`?
40. Why are relative paths important when building websites?

Part E: Practical questions

41. Write a complete HTML boilerplate.
42. Create an HTML page containing one heading and two paragraphs.
43. Create an unordered list of five programming languages.
44. Create an ordered list of five steps for completing a project.
45. Create a hyperlink to The Odin Project.
46. Display an image named `profile.jpg` stored inside an `images` folder.
47. Create a nested list containing programming languages and their frameworks.
48. Link `about.html` from `index.html` when both files are in the same directory.
49. Link from `pages/about.html` to `index.html` in its parent directory.
50. Describe the structure and functionality of the Odin Recipes project.

21. Practical Exercises

Exercise 1: Basic webpage

Create an HTML page containing:

- A complete HTML boilerplate.
- A title.
- One main heading.
- Three paragraphs.
- One strong element.
- One emphasis element.

Exercise 2: Shopping list

Create an unordered list containing five food items, three beverages and four household items.

Exercise 3: Daily routine

Create an ordered list describing your daily routine with at least five list items.

Exercise 4: Nested list

Create this nested structure:

- Frontend Development

- HTML

- CSS

- JavaScript

- Backend Development

- Node.js

- Express

- Databases

Exercise 5: Links

Create a webpage with links to The Odin Project, MDN Web Docs and GitHub.

Exercise 6: Images

Create a webpage displaying three images. Use relative paths and meaningful alternative text.

Exercise 7: Mini website

Create a three-page website:

```
mini-website/
■■■ index.html
■■■ about.html
■■■ contact.html
```

Requirements:

- Add navigation links between pages.
- Use headings and paragraphs.
- Add at least one image.
- Use a complete HTML boilerplate on every page.

22. Quick Revision Sheet

Concept	Remember
HTML	Structures webpage content
CSS	Styles webpage content
JavaScript	Adds functionality
Tag	Markup enclosed in angle brackets
Element	Complete HTML structure
Attribute	Additional element information
Void element	Element without a closing tag
DOCTYPE	Declares modern HTML document type
HTML root	`<html>`
Metadata	`<head>`
Visible content	`<body>`
Browser title	`<title>`
Character encoding	`<meta charset="UTF-8">`
Heading	`<h1>` to `<h6>`
Paragraph	`<p>`

Concept	Remember
Strong importance	`<strong>`
Stress emphasis	`<em>`
Comment	`<!-- comment -->`
Unordered list	`<ul>`
Ordered list	`<ol>`
List item	`<li>`
Hyperlink	`<a>`
Link destination	`href`
Image	`<img>`
Image location	`src`
Alternative text	`alt`
Current directory	`./`
Parent directory	`../`

Final HTML Checklist

- ■ Explain what HTML is.
- ■ Differentiate HTML, CSS and JavaScript.
- ■ Explain tags and elements.
- ■ Identify void elements.
- ■ Explain HTML attributes.
- ■ Write HTML boilerplate independently.
- ■ Explain the purpose of head and body.
- ■ Use headings and paragraphs correctly.
- ■ Understand strong and emphasis elements.
- ■ Nest HTML elements correctly.
- ■ Write HTML comments.
- ■ Create unordered and ordered lists.
- ■ Create nested lists.
- ■ Create internal and external links.
- ■ Differentiate absolute and relative URLs.
- ■ Display images using the img element.
- ■ Use image alternative text.
- ■ Understand relative file paths.
- ■ Build a multi-page HTML website.
- ■ Validate HTML code.

Final Thoughts

HTML is the foundation upon which the rest of web development is built. Understanding document structure, elements, attributes, nesting, lists, links and images will make CSS and JavaScript considerably easier to learn.

Learning sequence: HTML Structure → HTML Elements → Text → Lists → Links → Images → Practical Projects.

Learn the structure first. Style it later. Make it interactive after that.

Official Learning References

1. The Odin Project – Foundations: https://www.theodinproject.com/paths/foundations/courses/foundations
2. Introduction to HTML and CSS: https://www.theodinproject.com/lessons/foundations-introduction-to-html-and-css
3. Elements and Tags: https://www.theodinproject.com/lessons/foundations-elements-and-tags
4. HTML Boilerplate: https://www.theodinproject.com/lessons/foundations-html-boilerplate
5. Working with Text: https://www.theodinproject.com/lessons/foundations-working-with-text
6. Lists: https://www.theodinproject.com/lessons/foundations-lists
7. Links and Images: https://www.theodinproject.com/lessons/foundations-links-and-images
8. Project: Recipes: https://www.theodinproject.com/lessons/foundations-recipes
9. MDN HTML Documentation: https://developer.mozilla.org/en-US/docs/Web/HTML

Repository: The Odin Project – Web Dev Learning

Suggested file: `Foundations/04-HTML/notes.md`